# Sort the Vocabulary: Prefix-Ordered Token Classes Make Two-Level Language-Model Heads Cheap and Faithful

Lucas Tinoco*

October 2026

**Abstract**

In small language models with large subword vocabularies, the output head is a large share of the per-token cost: 21–63% of the weight bytes read per generated token in Gemma 3 270M, Qwen3-0.6B and Llama 3.2 1B. A two-level (class-factored) softmax reduces output-weight reads by approximately $\sqrt{V}/2$-fold, and has returned in 2025–26 in speculative-decoding drafters and drop-in head replacements; what remains open is *how to form the classes*. We show that the simplest possible rule—sort the subword vocabulary by spelling and cut it into equal blocks—gives the most faithful factored head among the standard grouping baselines in four of the five models tested. Swapping the head of five frozen open models (Gemma 3 270M/1B, Gemma 4 E2B, Qwen3-0.6B, gpt-oss-20b; three vendors, four tokenizers), among frequency, semantic, random and token-id baselines, prefix-ordered classes have the lowest KL divergence to the original model in four of five (semantic $k$-means has a slightly lower mean on the fifth), and 27–33% lower KL than the token ordering Google ships with its Gemma 4 drafters, at equal cost, including on Gemma 4 itself. Trained from scratch, prefix classes outperform the tested alternative groupings in recurrent and MLP models, with replication in English and a gap that shrinks as the vocabulary grows. A word-start/frequency and continuation/prefix hybrid further improves fidelity in four models at square-root-sized blocks. A Transformer with a prefix head is within 0.45% of the full head in bits per character at 4.2× fewer total MACs. A per-decision and word-start/continuation decomposition explains why: the first decision follows the generation direction, and the gain concentrates on tokens that continue a word—which also explains why alphabetical classes lost at the whole-word level (Jiang et al., 2017). On a CPU at batch size one, the factored head raises Gemma 3 270M throughput by 2.4× in end-to-end, one-token-at-a-time scoring (2.1× with AVX2 only).

## 1 Introduction

The output projection of a language model maps a hidden state $h \in \mathbb{R}^d$ to logits over a vocabulary of size $V$. Vocabularies have grown (128k in Llama 3, 152k in Qwen3, 201k in gpt-oss, 262k in Gemma) while the models people run locally have stayed small, so in small models the head is no longer a rounding error: it is 63% of the bytes read per token in Gemma 3 270M (Table 4). At batch size one—a model on a laptop or phone, or a draft model in speculative decoding—generation is memory-bound, and those bytes are time and energy.

Class-factored softmax [1, 2] is the classical remedy: predict a class, then a token inside the class, each with a small softmax; with $C \approx \sqrt{V}$ classes of $m \approx \sqrt{V}$ tokens the head reads $(C + m)d$

*Independent researcher. Implementation, experiments and drafting were carried out with the assistance of an AI coding agent (Claude); all results were run and checked on the author's machine. Code and raw results: `https://github.com/lucasrgt/prefix-head`.

instead of $Vd$ weights. It fell out of use on GPUs with large batches, where the full head is cheap, and because it costs quality. It is back: Gemma 4's drafters score 2,048 token clusters and expand the top 32 [10]; FlashHead replaces heads with balanced spherical $k$-means clusters and multi-probe retrieval [9]; CORAL factors a drafter head into equal groups [11]; FR-Spec and VocabTrim restrict drafter heads by frequency [12, 13]. All of these group tokens by embedding similarity, frequency, or token id.

We ask which grouping makes the factored head most faithful, at equal cost, and find that a rule needing no data and no model—*sort the vocabulary by spelling, cut into equal blocks*—outperforms standard grouping baselines in four models. Section 7 studies further improvements using a word-start/continuation hybrid. Our contributions:

- **Retrofit evidence on five frozen open models** (Section 3): among the standard grouping baselines, prefix classes give the lowest KL to the original head in four of five models (semantic classes have a slightly lower mean in the fifth), and beat the ordering shipped with Gemma 4 by 27–33% in KL at equal cost.
- **From-scratch evidence** across MLP, recurrent and Transformer models, two languages and vocabularies of 8k–100k (Section 4), including the honest finding that the gap shrinks with vocabulary size.
- **A mechanism** (Section 5): the class decision becomes easy because it follows the generation direction; the gain is 4–6× larger on word-continuation tokens. This reconciles our result with the opposite whole-word finding of Jiang et al. [5].
- **End-to-end CPU throughput and head-only GPU cost** at batch one (Section 6).
- **Variations** (Section 7): a word-start/continuation hybrid, controls that isolate the mechanism, three-level trees, block size, and a failed refinement.

## 2 Method

**Two-level head.** Given a partition of the vocabulary into classes $c(t)$ with members $M_c$, the head computes

$$\log p(t \mid h) = \log \operatorname{softmax}(Ah + b)_{c(t)} + \log \operatorname{softmax}\big(\ell_{M_{c(t)}}(h)\big)_t,$$

where $A \in \mathbb{R}^{C \times d}$ predicts the class and $\ell_{M_c}(h)$ are the original logits restricted to that class (including logit soft-capping where used). This defines a normalized distribution over the entire vocabulary: no token is excluded. It reproduces the full head only when the predicted class probabilities equal its class marginals. Sampling draws a class, then a token. At inference the head reads $C \cdot d$ class weights and one block of $m \cdot d$ token weights.

**Groupings compared.** *Prefix*: sort tokens by their byte spelling, cut into equal blocks of $m$. *Frequency*: sort by training frequency (the criterion of adaptive softmax and frequency-trimmed drafters). *Token id / merge order*: the tokenizer's own order (CORAL-style groups). *Random*. *Semantic*: balanced $k$-means on the output embeddings (as in FlashHead and, by our inspection, Gemma 4). *Gemma 4*: the `token_ordering` buffer released with the Gemma 4 E4B drafter, cut into its 2,048 blocks of 128.

Table 1: Fidelity of the factored head to the original model: $\mathrm{KL}(p_{\text{full}} \| p_{\text{2-level}})$ in bits/char on held-out text (lower is better). Prefix, semantic and the 128-block columns: mean ± sample s.d. over 3 seeds (2 for semantic on Gemma 3 270M); others: 1 seed.

| model (vocab, classes) | prefix | semantic | frequency | random | token id |
|---|---|---|---|---|---|
| Gemma 3 270M-it (262k, 512) | **0.1368**±0.0002 | 0.1621±0.0064 | – | – | – |
| Gemma 3 1B-pt (262k, 512) | 0.0867±0.0001 | **0.0854**±0.0018 | 0.133 | 0.192 | 0.214 |
| Gemma 4 E2B-it (262k, 512) | **0.1838**±0.0002 | 0.2738±0.0029 | 0.271 | 0.329 | 0.364 |
| Qwen3-0.6B (152k, 390) | **0.0958**±0.0001 | 0.1953±0.0032 | 0.133 | 0.177 | 0.200 |
| gpt-oss-20b (201k, 448) | **0.0197**±0.0001 | 0.0337±0.0024 | 0.024 | 0.042 | 0.037 |

*equal cost, 2,048 classes of 128: prefix vs. the ordering shipped with Gemma 4 (3 seeds)*

| model | prefix | Gemma 4 | prefix reduction |
|---|---|---|---|
| Gemma 3 270M-it | **0.1158**±0.0001 | 0.1593±0.0003 | 27% |
| Gemma 3 1B-pt | **0.0724**±0.0002 | 0.1086±0.0002 | 33% |
| Gemma 4 E2B-it | **0.1656**±0.0006 | 0.2414±0.0006 | 31% |

**Retrofitting a frozen model.** We keep the model and its output rows frozen and train only $A, b$. In the *self* setting the class predictor is trained to reproduce the model's own class marginals $p(c \mid h) = \sum_{t \in M_c} p_{\text{full}}(t \mid h)$ on training text (no labels, no other model), so its deviation from the full head is the pure cost of factoring. We report $\mathrm{KL}(p_{\text{full}} \,\|\, p_{\text{2-level}})$ in bits per character on held-out text; because $p(t \mid c, h)$ is exact, this equals the class-level KL. In the *adapt* setting both the full head and the two-level head (class predictor and blocks) are fine-tuned on the same text with the same budget. The self-trained predictor uses Adam for two epochs, batch size 512, initial learning rate $10^{-3}$ with linear decay, and class-centroid initialization; the output rows in the adapted heads use learning rate $10^{-4}$.

## 3 Retrofitting frozen models

**Setup.** Hidden states after the final norm are extracted with llama.cpp from 14 public-domain Portuguese novels (train 650–700k positions; test 714k characters, disjoint). Models: Gemma 3 270M-it and 1B-pt, Gemma 4 E2B-it (logit soft-capping 30 applied), Qwen3-0.6B, gpt-oss-20b (o200k tokenizer). Blocks of $m = \lceil \sqrt{V} \rceil$ (head cost cut 195–256×), plus $m = 128$ to match Gemma 4's ordering (120×).

**Results.** Table 1. Among the standard grouping baselines, prefix is the most faithful grouping in four of five models; on Gemma 3 1B semantic $k$-means is marginally better (0.0854 vs. 0.0867, within one s.d. of its seed spread), but semantic classes are worse than frequency on Qwen3 and Gemma 4. At equal cost prefix blocks deviate 27–33% less than Gemma 4's own ordering, including on Gemma 4.

**Bits per character, and a calibration caveat.** On base models the factored head costs a little: Gemma 3 1B-pt 1.561→1.615 bpc (+3.5%; 1.600 with 128-blocks), Qwen3-0.6B 1.918→1.942 (+1.2%), gpt-oss-20b 2.685→2.697 (+0.5%). On the instruction-tuned Gemmas the self-trained factored heads score *below* the full head (Gemma 4 E2B-it: 2.604→2.475), and the bpc ranking of groupings even inverts; the smoother class predictor partly corrects the instruction-tuned model's miscalibration on this out-of-domain text. We therefore use KL to the original model, not bpc, as the primary retrofit metric.

Table 2: From scratch, test bpc (lower is better). Means over seeds where given.

| model | prefix | semantic | merge order | frequency | random |
|---|---|---|---|---|---|
| MLP, BPE 8k (3 seeds) | **2.208** | 2.258 | 2.240–2.248 | 2.240–2.248 | – |
| Recurrent, BPE 8k (3 seeds) | **2.063** | 2.143 | 2.139[1] | 2.134[1] | 2.158[2] |
| Recurrent, English, BPE 8k | **1.762**[1] | – | 1.813[1] | – | – |
| MLP, English, BPE 8k (2 seeds) | **1.889** | – | 1.913 | 1.913 | – |

[1] one seed; [2] two seeds.

Table 3: Decomposition of test bits/char (recurrent model, BPE 8k): class decision + within-class decision, split by tokens that start a word and tokens that continue one.

| grouping | word start (527k chars) | continuation (189k chars) |
|---|---|---|
| prefix | 1.151 + 0.818 = **1.968** | 1.278 + 1.043 = **2.320** |
| merge order | 1.172 + 0.836 = 2.007 (+1.9%) | 1.641 + 0.863 = 2.504 (+7.9%) |
| semantic | 1.266 + 0.730 = 1.997 (+1.4%) | 1.953 + 0.578 = 2.531 (+9.1%) |
| frequency | 1.027 + 0.975 = 2.002 (+1.7%) | 1.542 + 0.959 = 2.501 (+7.8%) |
| second character | 1.220 + 0.773 = 1.994 (+1.3%) | 1.468 + 0.916 = 2.384 (+2.7%) |

**Equal-budget adaptation.** Fine-tuning both heads on the same text (Gemma 3 270M, Qwen3-0.6B): full 1.794 / 1.612 bpc; prefix 1.819 / 1.661 (+1.3% / +3.0%); semantic 1.842 / 1.751; frequency 1.896 / 1.724; random 1.872 / 1.723; token id 1.916 / 1.759. With 128-blocks on Gemma 3 270M: prefix +0.8%, Gemma 4 ordering +3.0%.

## 4 Training from scratch

We train small models on the same 14 novels with BPE vocabularies, the factored head with $C \approx m \approx \sqrt{V}$, and compare groupings at identical cost (raw runs: `results/alpha.jsonl` in the repository).

**Vocabulary size.** The advantage over frequency shrinks as the vocabulary grows: 3.5% (8k), 2.2% (32k), 1.1% (100k; merge order 3.8%, 2.3%, 1.4%). Larger vocabularies contain more whole-word tokens, and fewer word continuations, which is where prefix classes help (Section 5). The retrofit gaps on 152k–262k vocabularies are larger because a frozen body cannot adapt to the grouping.

**Transformer.** A 2-layer Transformer with the prefix head reaches 2.063 bpc against 2.054 with the full 8k head (+0.45%) at 4.2× fewer MACs per character.

## 5 Why prefix classes work

Two hypotheses could explain the effect. *Separability*—semantic classes force a hard choice between synonyms inside a class—is refuted: the within-class decision is *easier* for semantic classes (0.69 vs. 0.88 bits/char overall). *Direction* is supported: with prefix classes the first decision asks how the next piece of text begins, which the context predicts well, especially when a word is in progress.

Table 4: Output head as a share of the weight bytes read per generated token (Q8_0 GGUF, tied embeddings read as the head).

| model | vocab × $d$ | head params | head share of bytes/token |
|---|---|---|---|
| Gemma 3 270M | 262k × 640 | 168M of 268M | 63% |
| Gemma 3 1B | 262k × 1152 | 302M of 1.0B | 30% |
| Qwen3-0.6B | 152k × 1024 | 156M of 596M | 26% |
| Llama 3.2 1B | 128k × 2048 | 263M of 1.24B | 21% |
| SmolLM2-135M | 49k × 576 | 28M of 135M | 21% |

Table 5: Gemma 3 270M (Q8_0) end to end on CPU, batch one, tokens/s (mean of 3 runs of 512 tokens).

| ISA, threads | full head | prefix head (512×512) | speedup | full-head share of time |
|---|---|---|---|---|
| AVX-512, 8 | 132.0 | 321.2 | 2.4× | 56% |
| AVX-512, 1 | 75.5 | 183.5 | 2.4× | 57% |
| AVX2, 8 | 119.2 | 256.0 | 2.1× | 52% |
| AVX2, 1 | 58.2 | 120.5 | 2.1× | 52% |

Sorting by the second character instead gives an intermediate result. Table 3 localizes the gain: 4–6× larger on continuation tokens, and almost entirely in the class decision (1.28 vs. 1.54–1.95). With whole-word vocabularies every token starts a word, where the effect is small—consistent with Jiang et al. [5], where alphabetical classes lost to frequency and Brown clusters.

## 6 Cost

**GPU, head only.** On an RTX 4060 (fp16, CUDA graphs), scoring or sampling one token with the two-level head is 31–46× faster than the full head at batch one for the Llama 3.2 1B, Qwen3-0.6B and Gemma 3 270M head shapes, 20–31× at batch 8, and 2–10× at batch 512 with a naive per-row block gather.

**CPU, end to end.** We wrote a minimal Gemma 3 runtime in Rust (Q8_0 weights, AVX-512/AVX2, persistent thread pool) that matches llama.cpp to 0.3% in bpc over 700 tokens, and swapped only the head. Table 5 reports tokens/s on an idle Ryzen 7 7700 (Zen 4, 32 MB L3, dual-channel DDR5), 512 tokens of test text scored one at a time, mean of three runs. The full head takes 51–57% of each token's time; the prefix head (512 classes of 512) takes 1–2%. Whole-model throughput rises 2.4× with AVX-512 and 2.1× with AVX2, at 1 or 8 threads; with 2,048 classes of 128 the gain is 2.4× as well. Total package energy estimated from RAPL falls from approximately 0.59 to 0.25 J/token at 8 threads. These totals are reconstructed from the logged idle-subtracted energy by adding `idle_w/tok_per_s` per run; idle subtraction itself is unstable because the spinning thread pool keeps the package busy. Bits per character on these tokens: full 2.489, prefix head 2.435 (instruction-tuned model; see the calibration caveat above).

**With 4-bit weights.** Google's quantization-aware-trained release of Gemma 3 270M in Q4_0 keeps the tied 262k × 640 embedding/head in F16: 335 MB of a 399 MB file. Our runtime reads Q4_0 and F16 natively. With the full F16 head, the head takes 71% of each token's time (98

tokens/s, AVX-512, 8 threads); with the prefix head (512 classes of 512, class predictor trained on the QAT model's own states) throughput rises to 430 tokens/s (4.4×; run-to-run range 381–483), 3.4× the Q8_0 model with its full head (126 tokens/s in the same session). KL to the QAT model's own head is 0.120 bits/char (0.100 with 128-token blocks), comparable to the Q8 model. Weight quantization and head factorization compose. Requantizing the F16 head to 8 bits (absmax per 32 weights) costs nothing measurable (2.423 vs. 2.421 bits/char with the prefix head on 512 tokens) and shrinks the model from 393 to 235 MB; with it the prefix head reaches 537 tokens/s (5.5× the shipped QAT model, 4.3× the Q8_0 model with its full head). A 4-bit head saves a further 83 MB but costs 5% in bits/char. For reference, llama.cpp (commit a11f57b, 8 threads, full heads) generates 129 tokens/s with the QAT Q4_0 model and 175 tokens/s with the Q8_0 model on the same machine; our runtime is about 25–30% slower than llama.cpp with the full head, so the prefix-head figures are not explained by a faster body. Naively requantizing the non-QAT Q8 model to 4 bits instead costs 10% in bpc for the body alone, which is why we use the QAT release.

**A 1.58-bit model.** BitNet b1.58 2B4T [18] is trained with ternary weights, yet its tied 128k×2560 embedding/head is stored in F16: 657 MB against 523 MB for the whole ternary body in the released GGUF, 56% of the bytes read per token. We extracted its final hidden states with the reference implementation (online ternarization, `tools/gpu/hidden_hf.py`) and swapped the head as in Section 3 (358 classes). KL to the full head: prefix $0.1218 \pm 0.0001$ (3 seeds), frequency 0.125, token id 0.224, semantic $0.206 \pm 0.007$, random 0.264, and the word-start/continuation hybrid 0.110; bits per character 2.169 (full) → 2.193 (prefix). Reading only the class matrix and one block cuts per-token reads from about 1.18 GB to 0.53 GB, an upper bound of about 2.2× for memory-bound decoding; our runtime does not implement the BitNet architecture, so this is not measured end to end.

# 7 Variations

All numbers in this section are KL to the original model (bits/char, self setting, Section 3); prefix and hybrid rows are means over 3 seeds (s.d. $\leq 0.0003$), the rest single runs (`tools/gpu/explore.py`, `results/explore.jsonl`).

**Mixing criteria and controls.** Table 6. Ordering word-start tokens by frequency and word-continuation tokens by spelling (*hybrid*) improves on pure prefix in four models (by 2–11%) and is 0.6% worse on Gemma 4, at $\sqrt{V}$-sized blocks. With 128-token blocks the hybrid is worse than pure prefix on all three Gemmas (e.g. Gemma 3 1B: 0.0886 vs. 0.0724), though still better than the Gemma 4 ordering (0.1086). Three controls support the mechanism: ordering word starts semantically instead of by frequency is worse; ignoring the leading space, which merges word-start and continuation spellings, is worse; and sorting by the *reversed* spelling (suffix order) is far worse—similar spelling is not what matters, the beginning of the next piece is. Conversely, sorting under a randomly permuted byte alphabet (three permutations), which keeps every prefix family together but changes which families are neighbours, matches ordinary prefix order (Qwen3: 0.088–0.096 vs. 0.096; Gemma 3 1B: 0.086–0.089 vs. 0.087): the prefix tree matters, not the alphabet.

**Depth.** A three-level prefix tree (super-class, class, token; e.g. $64 \times 64 \times 64$ for 262k tokens) reads 808–1,365× fewer output rows than the full head. Its KL is 1.3–1.8× that of the two-level prefix head (Gemma 3 1B: 0.124 vs. 0.087; Qwen3: 0.127 vs. 0.096), and lower than that of the same three-level structure over a random order in every model (Gemma 4 E2B: 0.262 vs. 0.493; gpt-oss:

Table 6: Grouping variants at $\sqrt{V}$-sized blocks (KL, lower is better).

| variant | G3 270M | Qwen3 0.6B | G3 1B | G4 E2B | gpt-oss 20b |
|---|---|---|---|---|---|
| prefix | 0.137 | 0.096 | 0.087 | **0.184** | 0.020 |
| hybrid: starts by frequency, continuations by prefix | **0.126** | **0.091** | **0.077** | 0.185 | **0.019** |
| starts by semantic $k$-means, continuations by prefix | 0.144 | 0.107 | 0.090 | 0.204 | 0.021 |
| prefix ignoring the leading space | 0.157 | 0.102 | 0.100 | 0.199 | 0.025 |
| suffix (reversed spelling) | 0.197 | 0.141 | 0.139 | 0.257 | 0.030 |

0.036 vs. 0.056). The relative advantage over random order increases with depth on the Gemmas and Qwen3, but decreases on gpt-oss-20b; its dependence on depth is therefore model-specific. Because the two-level head already takes only 1–2% of a token's time on CPU (Section 6), deeper heads matter mainly for drafters whose body is tiny next to a 262k vocabulary.

**Granularity.** Prefix blocks have lower KL than random blocks at every block size with a matched random control. Prefix-only sweeps cover block sizes from 64 to 4,096 (up to 1,024 for gpt-oss-20b); random controls are available at 128, approximately $\sqrt{V}$, and 2,048 (the first two for gpt-oss-20b). The shape of the cost–fidelity curve is model dependent: for the Gemmas and Qwen3, many small classes beat few large ones in the tested configurations (Qwen3 at a 116× vs. 72× cut: 0.075 with 128-token blocks vs. 0.101 with 2,048-token blocks), whereas gpt-oss-20b becomes *less* faithful with small blocks (0.037 at 64 vs. 0.020 at 449).

**Speculative-decoding acceptance.** Gemma 3 270M as drafter, Gemma 3 1B and Gemma 4 E2B as targets (identical tokenizations, 214k positions). We report the per-position acceptance of a sampled draft token, $\alpha = \sum_t \min(p_t, q_t)$, a teacher-forced proxy for speculative decoding [17]. With the exact two-level head and 128-token blocks (2,176 rows read), prefix blocks keep $\alpha = 0.477$ / 0.371 (targets 1B / E2B) against 0.460 / 0.345 for the Gemma 4 ordering and 0.490 / 0.396 for the full drafter head: prefix loses about half as much acceptance. Gemma 4's own scheme—score the classes, keep the top 32, softmax over their tokens (6,144 rows)—is nearly lossless with any ordering (0.453–0.488 / 0.397–0.398), and with it the Gemma 4 ordering is better than prefix on the 1B target (0.480 vs. 0.453); under that scheme greedy acceptance is the same for every ordering (0.447 / 0.384). So for drafters verified by a target, top-$k$ screening makes the ordering nearly irrelevant, and prefix ordering does not improve it; the exact prefix head reaches similar acceptance on the 1B target while reading 2.8× fewer rows, and remains the option for standalone models that need probabilities for every token.

**Exact culling.** We also tried to find the exact argmax token while reading fewer rows, with bounding spheres per block (centre plus radius, as in ray-tracing BVHs) and with a low-precision copy of the head whose per-row error bound decides which rows to recompute. At $d = 640$–$1024$ both bounds are too loose: on Gemma 3 270M and Qwen3 essentially every block is visited and almost every row refined, for prefix, random and semantic blocks alike.

**Refinement.** Starting from prefix blocks, we moved each token to the class whose trained predictor already assigns high probability in that token's contexts (capacity-limited) and retrained, for three rounds. Every round was worse than the prefix start in every model (e.g. Qwen3: 0.096 → 0.167, 0.132, 0.145), so this heuristic does not improve on spelling order; whether a direct KL-optimizing search can is left open.

## 8 Related work

Class-based and hierarchical softmax [1, 3, 2]; adaptive softmax [4]; tree-based hierarchical softmax with alphabetical, Huffman and Brown trees at the word level [5]; prefix-tree lexicons for speech decoding search [6]. Screening and approximate heads: SVD-softmax [7], L2S [8], FlashHead [9]. Drafter heads: Gemma 4 [10], CORAL [11], FR-Spec [12], VocabTrim [13]. Training-memory reduction of large heads without changing inference cost: Cut Cross-Entropy [14]. To our knowledge no prior work orders a subword vocabulary by spelling to form output classes.

## 9 A frugal recurrent body (secondary study)

For CPU deployment we pair the head with a diagonal complex linear recurrence $h_t = (1 - z_t)\, e^{i\theta} h_{t-1} + z_t c_t$ with input-only gates and a learned, data-independent rotation. We arrived at it independently, but it is the HGRU recurrence of HGRN [19]; our differences are a chrono-style log-uniform initialization of gate biases and stateful truncated-BPTT training. At tiny scale (2.2M parameters, 2 layers, 51 complex channels per layer) it matches a 2-layer Transformer with the same head (2.063 vs. 2.063 bpc) at 3.6× fewer MACs per character and an 816-byte inference state instead of a growing 128 KB KV cache, and beats our Mamba-2-style baseline (2.078 bpc at 1.6× the MACs). Removing the rotation ($\theta = 0$, so the real and imaginary halves become $2 \times 51$ independent real channels in the same 816 bytes) costs 0.8%: 2.079 bpc (2 seeds: 2.0799, 2.0789) against 2.063 (3 seeds: 2.0614, 2.0665, 2.0608). At this scale the rotation helps. Letting part of the channels update only every $k$ positions and hold their state in between (Clockwork-style; their read-out is then constant and can be cached) trades a little quality for fewer MACs: over 3 seeds, 80 channels with 50 updating every 8 positions cost +0.26% bpc for 21% fewer MACs per character, and 51 channels with 26 slow ones +0.46% for 33% fewer. Spreading the periods over powers of two (1 to 64, one seed) did not improve this trade-off. We also tried a synthetic associative-recall task (2–32 key–value pairs per line, 2M characters, 30 epochs): neither the recurrent core with or without rotation nor a same-width Transformer learned it within that budget (recurrent: 3–5% accuracy on recalled characters, chance 3.8%; Transformer: 5–7%, declining with the number of pairs), so this probe is inconclusive and the known recall weakness of fixed-size states [15, 16] remains untested here.

## 10 Limitations

One text domain (19th-century public-domain literature) for all retrofit and most scratch results, plus English for replication. Retrofit models up to 21B parameters (3.6B active); scratch models are tiny. We evaluate likelihood, KL and a teacher-forced acceptance proxy, not downstream tasks or wall-clock speculative decoding. The factored head does not reduce the stored model size, only the bytes read per token. Gains at large batch on GPUs are modest.

## Acknowledgments

Thanks to Gabriel Ribeiro for many ideas along the way, and to Claude (Anthropic), the AI assistant that helped implement, run and write up this work.

## References


[1] J. Goodman. Classes for fast maximum entropy training. ICASSP, 2001.

[2] T. Mikolov, S. Kombrink, L. Burget, J. Černocký, S. Khudanpur. Extensions of recurrent neural network language model. ICASSP, 2011.

[3] F. Morin, Y. Bengio. Hierarchical probabilistic neural network language model. AISTATS, 2005.

[4] E. Grave, A. Joulin, M. Cissé, D. Grangier, H. Jégou. Efficient softmax approximation for GPUs. ICML, 2017.

[5] N. Jiang, W. Rong, M. Gao, Y. Shen, Z. Xiong. Exploration of tree-based hierarchical softmax for recurrent language models. IJCAI, 2017.

[6] H. Ney, R. Haeb-Umbach, B.-H. Tran, M. Oerder. Improvements in beam search for 10000-word continuous speech recognition. ICASSP, 1992.

[7] K. Shim, M. Lee, I. Choi, Y. Boo, W. Sung. SVD-Softmax: fast softmax approximation on large vocabulary neural networks. NIPS, 2017.

[8] P. H. Chen, S. Si, S. Kumar, Y. Li, C.-J. Hsieh. Learning to screen for fast softmax inference on large vocabulary neural networks. ICLR, 2019.

[9] W. Tranheden, S. Ahmed, D. Dubhashi, J. Matthiesen, H. von Essen. FlashHead: efficient drop-in replacement for the classification head in language model inference. arXiv:2603.14591, 2026.

[10] Gemma Team. Gemma 4 technical report. arXiv:2607.02770, 2026.

[11] Y. Weng et al. CORAL: learning consistent representations across multi-step training with lighter speculative drafter. arXiv:2502.16880, 2025.

[12] W. Zhao, T. Pan, X. Han et al. FR-Spec: accelerating large-vocabulary language models via frequency-ranked speculative sampling. ACL, 2025.

[13] R. Goel et al. VOCABTRIM: vocabulary pruning for efficient speculative decoding in LLMs. arXiv:2506.22694, 2025.

[14] E. Wijmans, B. Huval, A. Hertzberg, V. Koltun, P. Krähenbühl. Cut your losses in large-vocabulary language models. ICLR, 2025.

[15] S. Arora et al. Zoology: measuring and improving recall in efficient language models. arXiv:2312.04927, 2023.

[16] S. Jelassi, D. Brandfonbrener, S. M. Kakade, E. Malach. Repeat after me: Transformers are better than state space models at copying. arXiv:2402.01032, 2024.

[17] Y. Leviathan, M. Kalman, Y. Matias. Fast inference from Transformers via speculative decoding. ICML, 2023.

[18] S. Ma, H. Wang et al. BitNet b1.58 2B4T technical report. arXiv:2504.12285, 2025.

[19] Z. Qin, S. Yang, Y. Zhong. Hierarchically gated recurrent neural network for sequence modeling. NeurIPS, 2023.